

2025-2 마이크로 전자회로 중간고사

다음은 자판기를 구현하는 프로그램에 대한 내용이며, 각 문제의 답안은 다음 문제 풀이에 활용됨.

문제 1) 모든 판매 물품의 기본이 될 **Item** 클래스를 작성

- 이 클래스의 객체는 생성될 때 **name** (이름)과 **price** (가격) 정보를 받아서 속성으로 저장해야 함.

```
# 문제 1: 전체 실행 코드
class Item:
    def __init__(self, name, price):
        # TODO: self를 사용하여 name과 price 속성을 초기화하세요.
        self.name = name
        self.price = price

# --- 실행 및 테스트 부분 ---
# 아래 코드를 실행했을 때, 오류 없이 결과가 출력되어야 합니다.
if __name__ == "__main__":
    item1 = Item("과자", 1500)
    print("✅ 문제 1 테스트")
    print(f"이름: {item1.name}, 가격: {item1.price}원")
    print("-" * 20)
```

출력 결과 :

```
✅ 문제 1 테스트
이름: 과자, 가격: 1500원
-----
```

문제 2) 문제 1에서 만든 **Item** 클래스를 상속받는 **Drink** 클래스 작성

- `Drink` 클래스는 `name`, `price` 속성 외에 `volume` (용량)이라는 추가 속성을 가져야 함.
- `super()` 를 사용하여 부모 클래스의 `__init__` 메소드를 호출

문제 2: 전체 실행 코드

(문제 1에서 완성한 Item 클래스)

class Item:

def __init__(self, name, price):

self.name = name

self.price = price

class Drink(Item):

def __init__(self, name, price, volume):

TODO: super()를 사용하여 부모 클래스의 __init__을 호출하고,

volume 속성을 추가로 초기화하세요.

super().__init__(name, price)

self.volume = volume

--- 실행 및 테스트 부분 ---

if __name__ == "__main__":

drink1 = Drink("콜라", 1800, "500ml")

print("✅ 문제 2 테스트")

print(f"이름: {drink1.name}, 가격: {drink1.price}원, 용량: {drink1.volume}")

print("-" * 20)

출력 결과 :

✅ 문제 2 테스트

이름: 콜라, 가격: 1800원, 용량: 500ml

문제 3) `VendingMachine` (자판기) 클래스 작성

- 생성될 때 `Item` 또는 `Drink` 객체들이 담긴 `리스트`를 재고(`inventory`)로 처리.
- `display_menu()` 메소드는 재고 상품의 이름과 가격을 번호와 함께 출력.

문제 3: 전체 실행 코드

import random # 이 문제는 random 모듈을 사용하지 않습니다.

(문제 1, 2에서 완성한 클래스)

class Item:

def __init__(self, name, price):

self.name = name

self.price = price

class Drink(Item):

def __init__(self, name, price, volume):

super().__init__(name, price)

self.volume = volume

class VendingMachine:

def __init__(self, inventory):

TODO: inventory 속성을 초기화하세요.

self.inventory = inventory

def display_menu(self):

print("--- 자판기 메뉴 ---")

TODO: for 반복문과 enumerate를 사용하여

재고 목록의 모든 아이템을 번호, 이름, 가격 형식으로 출력하세요.

(예: 1. 콜라 (1800원))

for i, item in enumerate(self.inventory):

print(f"{i + 1}. {item.name} ({item.price}원)")

print("-----")

--- 실행 및 테스트 부분 ---

if __name__ == "__main__":

다양한 종류의 재고 생성

coke = Drink("콜라", 1800, "500ml")

water = Drink("물", 1000, "450ml")

snack = Item("감자칩", 1500)

vm = VendingMachine([coke, water, snack])

```
print("✅ 문제 3 테스트")
vm.display_menu()
```

출력 결과 :

```
✅ 문제 3 테스트
--- 자판기 메뉴 ---
1. 콜라 (1800원)
2. 물 (1000원)
3. 감자칩 (1500원)
-----
```

문제 4) 문제 3의 `VendingMachine` 클래스에 `sell()` 메소드를 추가

- 예외 처리를 사용하여 `ValueError` (숫자 아닌 값 입력)와 `IndexError` (목록에 없는 번호 입력) 상황을 처리.

문제 4: 전체 실행 코드

(문제 1, 2에서 완성한 클래스)

```
class Item:
```

```
    def __init__(self, name, price):
```

```
        self.name = name
```

```
        self.price = price
```

```
class Drink(Item):
```

```
    def __init__(self, name, price, volume):
```

```
        super().__init__(name, price)
```

```
        self.volume = volume
```

(문제 3에서 완성한 `VendingMachine` 클래스에 `sell` 메소드 추가)

```
class VendingMachine:
```

```
    def __init__(self, inventory):
```

```
        self.inventory = inventory
```

```
    def display_menu(self):
```

```
        print("--- 자판기 메뉴 ---")
```

```
        for i, item in enumerate(self.inventory):
```

```

        print(f"{i + 1}. {item.name} ({item.price}원)")
    print("-----")

def sell(self):
    # TODO: 이 메소드를 완성하세요.
    self.display_menu()
    try:
        choice = input("음료 번호를 입력하세요: ")
        # 1. 입력받은 choice를 정수(int)로 변환
        choice_num = int(choice)
        # 2. 리스트 인덱스로 변환 (사용자는 1부터 시작)
        item_index = choice_num - 1

        # 3. 인덱스 범위 확인
        if item_index < 0: # 0 또는 음수를 입력한 경우도 IndexError와 동일하게
처리
            raise IndexError

        # 4. 선택된 아이템 가져오기 및 출력
        selected_item = self.inventory[item_index]
        print(f"{selected_item.name}을(를) 선택했습니다.")

        # TODO: except ValueError 블록을 추가하여 "숫자를 입력해야 합니다." 출력
    except ValueError:
        print("숫자를 입력해야 합니다.")
        # TODO: except IndexError 블록을 추가하여 "목록에 없는 번호입니다." 출력
    except IndexError:
        print("목록에 없는 번호입니다.")

# --- 실행 및 테스트 부분 ---
if __name__ == "__main__":
    coke = Drink("콜라", 1800, "500ml")
    snack = Item("감자칩", 1500)
    vm = VendingMachine([coke, snack])

    print("✅ 문제 4 테스트")
    print("👉 시나리오 1: 정상 입력 (2 입력)")

```

```

vm.sell()
print("\n👉 시나리오 2: 잘못된 번호 입력 (3 입력)")
vm.sell()
print("\n👉 시나리오 3: 문자 입력 ('가나다' 입력)")
vm.sell()

```

출력 결과 :

```

✅ 문제 3 테스트
--- 자판기 메뉴 ---
1. 콜라 (1800원)
2. 물 (1000원)
3. 감자칩 (1500원)
-----

```

문제 5) 다형성을 활용하여 코드를 최종 완성하시오.

- `Item` 과 `Drink` 클래스에 각각 정보를 출력하는 `show_info()` 메소드를 추가/오버라이딩.
- `VendingMachine` 의 `display_menu()` 가 각 객체의 `show_info()` 를 호출하도록 수정.

문제 5: 전체 실행 코드

```

class Item:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    # TODO: Item의 정보를 출력하는 show_info 메소드 추가
    # 출력 형식: "이름: 감자칩, 가격: 1500원"
    def show_info(self):
        print(f"이름: {self.name}, 가격: {self.price}원")

class Drink(Item):
    def __init__(self, name, price, volume):
        super().__init__(name, price)
        self.volume = volume

```

```

# TODO: Drink의 정보를 출력하도록 show_info 메소드 오버라이딩
# 출력 형식: "이름: 콜라, 가격: 1800원, 용량: 500ml"
def show_info(self):
    print(f"이름: {self.name}, 가격: {self.price}원, 용량: {self.volume}")

class VendingMachine:
    def __init__(self, inventory):
        self.inventory = inventory

    # TODO: display_menu 메소드가 각 item의 show_info()를 호출하도록 수정
    def display_menu(self):
        print("--- 자판기 메뉴 ---")
        for item in self.inventory:
            # item이 Item 객체이든 Drink 객체이든 알아서 맞는 show_info가 실행됨
            # (다형성)
            item.show_info()
        print("-----")

    # sell 메소드는 수정할 필요 없습니다.

# --- 실행 및 테스트 부분 ---
if __name__ == "__main__":
    coke = Drink("콜라", 1800, "500ml")
    snack = Item("감자칩", 1500)
    vm = VendingMachine([coke, snack])
    print("✅ 문제 5 테스트")
    vm.display_menu()

```

출력 결과 :

```

✅ 문제 5 테스트
--- 자판기 메뉴 ---
이름: 콜라, 가격: 1800원, 용량: 500ml
이름: 감자칩, 가격: 1500원
-----

```